Padrões GoF – Singleton

Sabrina de F. Souto.

QUESTÕES INICIAIS

- O que é uma classe?
- O que é uma instância (de uma classe)?
- O que significa acesso definido pelos escopo?
- O que é acesso global?
- Como o acesso global é possível em java?

O QUE SE DESEJA?

- Garantir que uma classe possua apenas uma instância
- Proporcionar acesso global a esta instância

- Para quê?
  - Proporcionar a criação de classes Factory
    - Veremos isto mais adiante
  - Controlar a criação de objetos caros e pesados
  - Tornar as interfaces dos módulos da arquitetura únicas e acessíveis de forma global.

PROBLEMA

- Como assegurar que uma classe tenha uma **única instância** e prover um **ponto de acesso global** a esta instância?

SOLUÇÕES

- O que queremos?
  - Instância Única
  - Acesso Global

- **Solução 01**
  - Uso de variáveis globais
  - O que são variáveis globais?
    - Como as definimos em java?
  - Problema?
    - Uma variável global deixa a instância acessível mas não inibe a instanciação múltipla

SOLUÇÕES

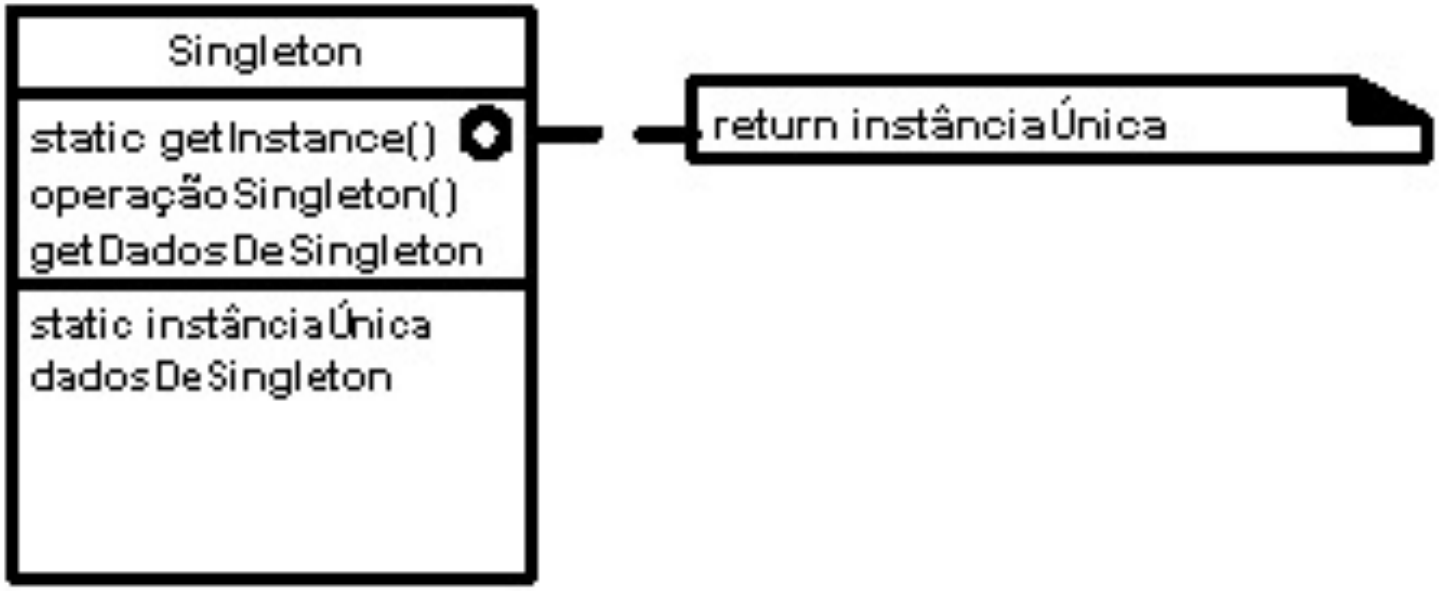
- O que queremos?
  - Instância Única
  - Acesso Global

- **Solução 02**
  - Use o padrão Singleton
  - Faça com que a classe em si seja responsável pela criação e manutenção da sua instância única

SINGLETON

- Deve haver uma **única instância** de uma classe e esta instância deve ser acessada a partir de um **ponto de acesso bem-conhecido**

ESTRUTURA


Singleton
static getInstance()
operaçãoSingleton()
getDadosDeSingleton
static instânciaÚnica
dadosDeSingleton
return instânciaÚnica

PARTICIPANTES

- **Singleton**
  - Define uma operação **getInstance()** que permite que clientes acessem sua instância única
    - É um método estático
      - permite acesso sem que seja necessário criar a classe na qual este se encontra
  - Deve ser responsável pela criação de sua instância única

COLABORAÇÕES

- Clientes acessam a instância apenas através da operação **getInstance()** do Singleton
  - O que é um cliente?
  - Sempre que quiserem uma instância da classe podem obtê-la a partir da chamada ao métodos **getInstance()**
  - Por se tratar de um método estático, a chamada ao mesmo se dá:
    - NomeDaClasse.nomeDoMetodo()

CONSIDERAÇÕES DE IMPLEMENTAÇÃO

- Como garantir uma instância única?
  - Não permitir chamadas ao construtor
  - Construtor private
  - A classe possui um objeto (*static*) dela mesma.

CONSIDERAÇÕES DE IMPLEMENTAÇÃO

```
public class Config {

    private static Config instanciaUnica = null;

    private Config() {

    }

    public static Config getInstance() {

        if(instanciaUnica == null) {
            instanciaUnica = new Config(); "
        }

        return instanciaUnica;
    }
}
```

CONSEQUÊNCIAS

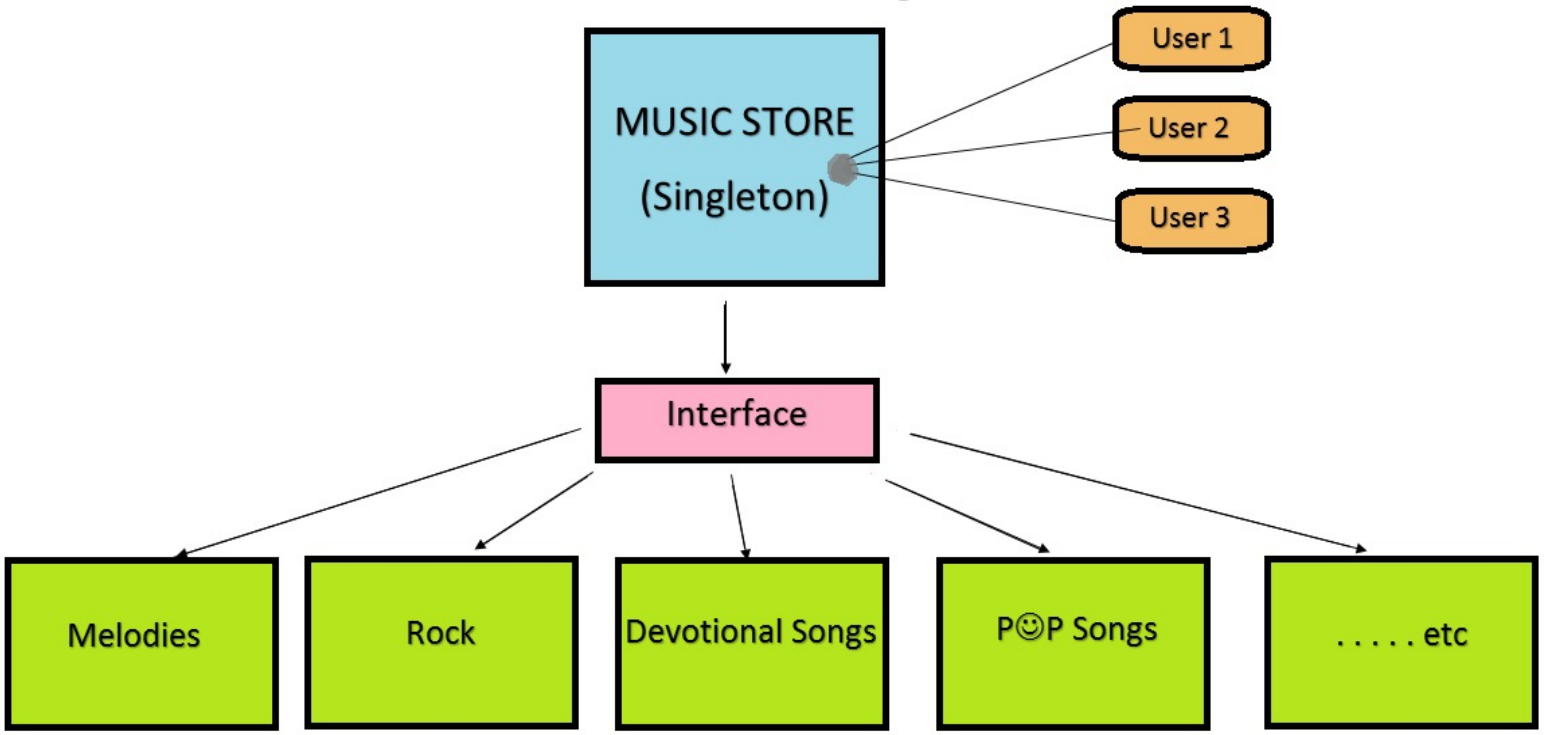
- Acesso controlado à instância única
  - O singleton tem controle sobre como e quando clientes acessam a instância

- Permite refinamento de operações e de representação
  - Várias classes Singleton podem obedecer a mesma interface, permitindo que um singleton particular seja escolhido para trabalhar com uma determinada aplicação em tempo de execução

CONSEQUÊNCIAS

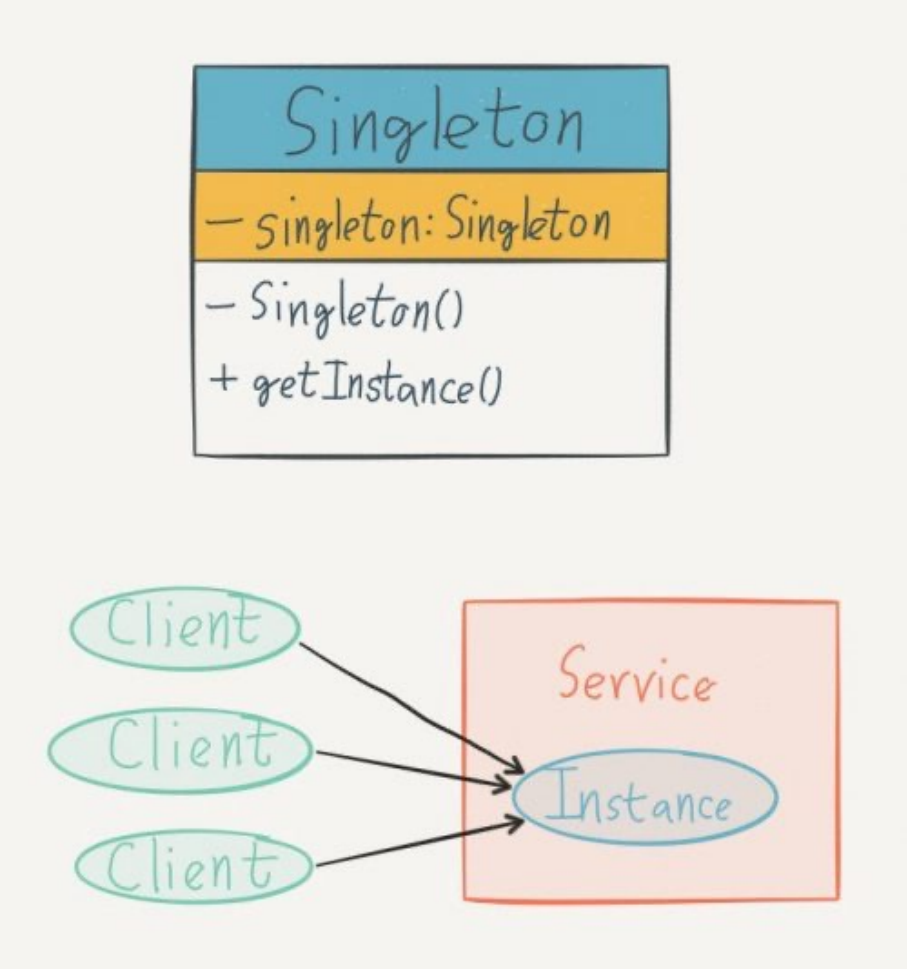
- Permite a existência de um número variável de instâncias
  - É fácil fazer com que o Singleton crie um número fixo, ou um número máximo de instâncias em vez de apenas uma única instância

SINGLETON – APLICAÇÕES



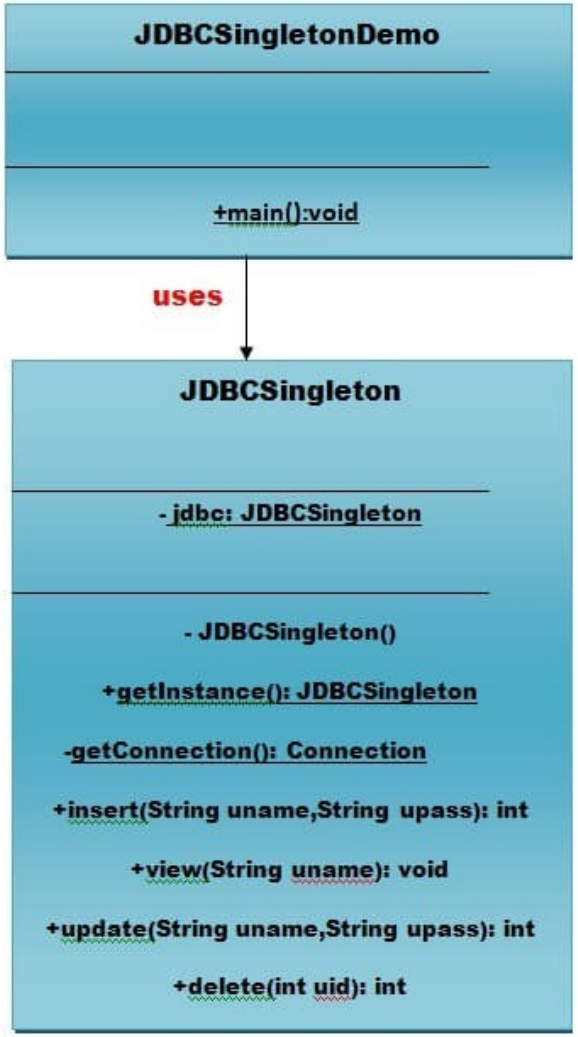
FONTE: https://aishwaryavaishno.wordpress.com/2013/05/17/global-access-point-singleton-design-pattern/

SINGLETON – APLICAÇÕES



FONTE: https://girlsincode.com/javascript/javascript-design-patterns-singleton-and-modules/

SINGLETON – APLICAÇÕES



FONTE: https://www.javatpoint.com/singleton-design-pattern-in-java

REFLEXÃO

- Como permitir que tenhamos um número X de instâncias de uma determinada classe?

- Qual o tratamento que deve ser dado ao tentar acessar uma instância já existente?

Dúvida?